

Array e Hashing

Monday, April 27, 2026 9:50 AM

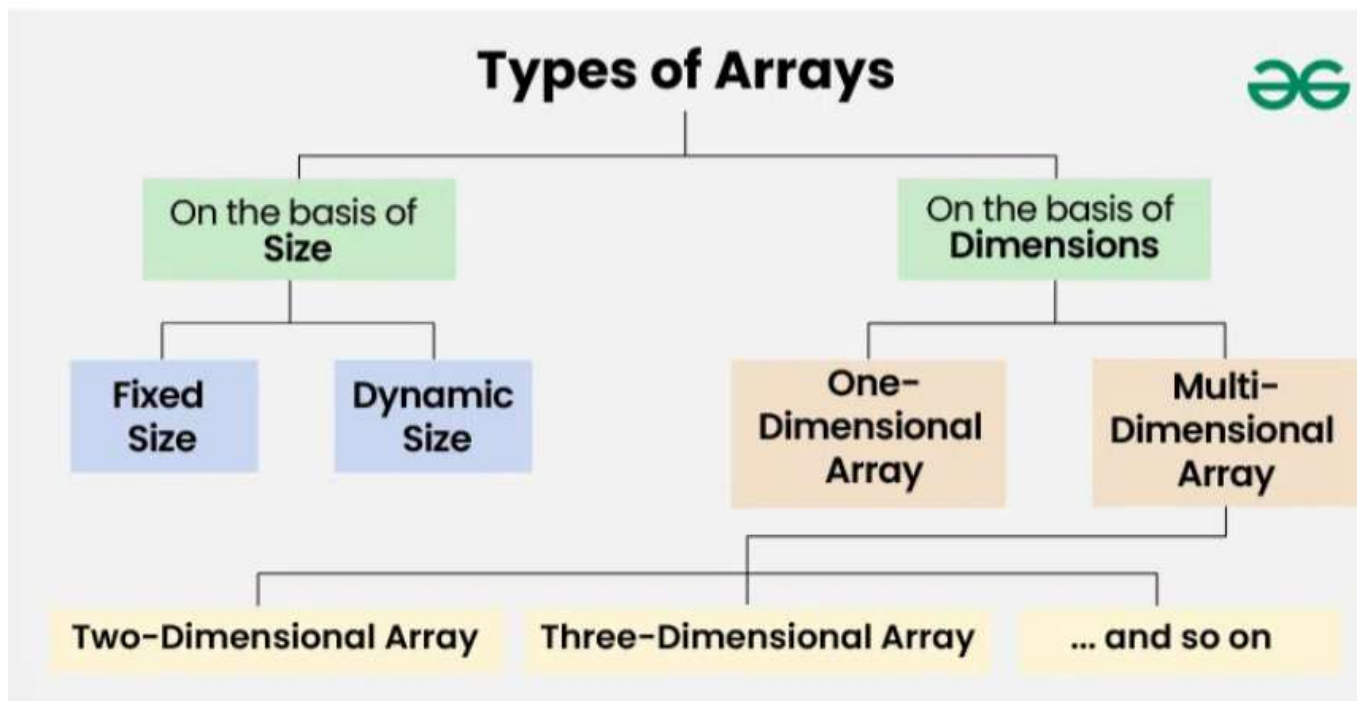
ARRAY

An array is a collection of items of the same variable type that are stored at contiguous memory locations.

Arr = []

Arrays can be classified in two ways:

- On the basis of size
 - o Fixed size arrays
 - o Dynamic sized arrays
- On the basis of dimensions
 - o One-dimensional array (1D): as a row where elements are store one after another.
 - o Two-dimensional array (2D): an array of arrays or a matrix composed of rows and columns.
 - o Three-dimensional array: array of matrices.



Operations on Array

- Traversal in array: process of accessing each element in the array sequentially, typically to perform a specific operation, such as searching, sorting or modifying elements.

Types:

- o Linear traversal: process of visiting each element of an array sequentially, starting from the first element and moving to the last element.

```
For i in arr:  
    Print(i)
```

Time complexity: $O(n)$

Space: $O(1)$

- o Reverse traversal: is the process of visiting each element starting from the last element and moving towards the first element.

```
For i in range(len(arr)-1, -1, -1):  
    Print(arr[i])
```

Time complexity: $O(n)$

Space: $O(1)$

Methods:

- For
- While:
 While $i < n$:
 Print(arr[i])
 $i += 1$
- For each:
 For value in arr:
 Print(value)

Applications:

- Searching:
 For i in range(len(arr)):
 If arr[i] == target:
 Found = true
 Break
- Modifying elements
 For i in range(len(arr)):
 Arr[i] += 5

- Insertions

- At the beginning of an array:
 - Built-in methods: arr.insert(0, element). Time complexity: $O(n)$
 - Custom methods:
 For i in range(n-1, -1, -1):
 Arr[i+1] = arr[i] #shift to the right
 Arr[0] = element
 Time complexity: $O(n)$
- At a given position:
 - Built-in methods: arr.insert(pos-1, element). Time complexity: $O(n)$
 - Custom methods:
 For i in range(n, pos-1, -1):
 Arr[i] = arr[i-1]
 Arr[pos-1] = element
 Time complexity: $O(n)$
- At the end:
 - Built-in methods: arr.append(element). Time complexity: $O(1)$
 - Custom methods: arr[n] = element. Time complexity: $O(1)$

- Deletion:

- From the beginning:
 - Built-in methods: del arr[0]. Time complexity: $O(n)$
 - Custom methods:
 For i in range(1, n):
 Arr[i-1] = arr[i]
 $N -= 1$
 Time complexity: $O(n)$
- From a given position:
 - Built-in methods: del arr[pos-1]. Time complexity: $O(n)$
 - Custom methods:
 For i in range(pos, n):
 Arr[i-1] = arr[i]
 If pos <= n:
 $n -= 1$
 Time complexity: $O(n)$
- Delete first occurrence of a given element:
 - Built-in methods:
 If ele in arr: _____

- ```

 Arr.remove(ele)
 Time complexity: O(n)
 ▪ Custom methods:
 Found = False
 For i in range(n):
 If found:
 Arr[i-1] = arr[i]
 Elif arr[i] == ele:
 Found = true
 If found:
 n-=1
 Time complexity: O(n)
 ○ Remove all occurrences of an element:
 Def remove_element(arr: List[int], ele:int) -> int:
 k=0
 For i in range(len(arr)):
 If arr[i]!=ele:
 Arr[k], arr[i] = arr[i], arr[k]
 K +=1
 Return k
 Def main():
 Arr = [...]
 Ele = 2
 Print(remove_element(arr, ele))
 Time complexity: O(n)
 ○ Delete from the end of the array:
 ▪ Built-in methods: arr.pop(). Time complexity: O(1)
 ▪ Custom methods: n -=1. Time complexity: O(1)
- Searching
 ○ Linear search
 Def search(arr, x):
 N = len(arr)
 For i in range(0, n):
 If(arr[i] == x):
 Return i
 Return -1
 Time complexity:
 ▪ Best case: the key might be present at first index: O(1)
 ▪ Worst case: in the last idnex: O(N)
 ▪ Average case: O(N)
 Applications:
 ▪ Unsorted list
 ▪ Small datasets
 ▪ Searching linked lists
 ▪ Simple implementation
 ○ Binary search
 Conditions to be applied:
 ▪ Data structure must be sorted
 ▪ Access to any element of the data structure should take constant time
 It can be implemented in both iterative and recursive ways.
 ▪ Iterative:
 Def search(arr, x):
 Low = 0
 High=len(arr)-1
 While lox <= high:
 Mid = low + (high-low)//2
 If arr[mid] == x:
 Return mid

```

```

 Elif arr[mid] < x:
 Low = mid+1
 Else:
 High = mid-1
 Return -1

```

Time complexity:  $O(\log n)$   
 Space complexity:  $O(1)$

- Recursive:
 

```

 Def search(arr, low, high, x):
 If high >= low:
 Mid = low + (high-low)//2
 If arr[mid] == x:
 Return mid
 Elif arr[mid] > x:
 Return search(arr, low, mid-1, x)
 Else:
 Return search(arr, mid+1, high, x)
 Else:
 Return -1

```

Time complexity:  $O(\log n)$   
 Space complexity:  $O(\log n)$

In the best case the time complexity is  $O(1)$ , the worst and average case are the same, equal to  $O(\log N)$ . The space is  $O(1)$ , but if the recursive call stack is considered it becomes  $O(\log N)$ .

## STRINGS

They are sequences of characters. A string is terminated with a special character `'\0'` and are immutable.

`S = "CIAO"`

General operations:

- Length of string: `len(s)`. Time complexity:  $O(n)$
- Search a character:
  - o By traversing the string. Time complexity:  $O(n)$

```

def findChar(s, ch):
 N = len(s)
 For i in range(n):
 If s[i] == ch:
 Return i
 Return -1

```

- o Built-in methods. Time complexity:  $O(n)$

```

Def find_char_index(s, ch):
 Idx = s.find(ch)
 Return idx

```

- Check for substring
  - o Nested loops. Time complexity:  $O(m*n)$

```

Def findSubstring(txt, pat):
 N = len(txt)
 M = len(pat)
 For i in range(n-m+1):
 J=0
 While j<m and text[i+j] == pat[j]:
 j+=1
 If j==m:
 Return i
 Return -1

```

- o Pattern searching algorithms: see <https://www.geeksforgeeks.org/dsa/introduction-to-pattern-searching/>
- o Built-in methods:

- ```
Idx = txt.find(pat)
```
- Insert a character
 - o Built-in methods: time complexity $O(n)$

```
Def insertChar(s, c, pos):
    Return s[:pos] + c + s[pos:]
```
 - o Custom methods: time complexity $O(n)$

```
Def insertChar(s, c, pos):
    Res = " "
    For i in range(len(s)):
        If i == pos:
            Res += c
        Res += s[i]
    If pos >= len(s):
        Res += c
    Return res
```
 - o Delete a character
 - Loop: time complexity $O(n)$, space $O(n)$

```
New = " "
For c in s:
    If c != char:
        New += c
Return new
```
 - Built-in methods: time complexity $O(n)$, space $O(n)$

```
New = str[:pos] + str[pos+1:]
```
 - o Check for same strings: == operator. Time complexity $O(n)$, space $O(1)$
 - o String concatenation: using + operator. Time complexity $O(n+m)$, space $O(n+m)$
 - o Reverse a string
 - Backward traversal: time complexity $O(n)$, space $O(n)$

```
Res = []
For i in range(len(s)-1, -1, -1):
    Res.append(s[i])
Return ".join(res)
```
 - Using two pointers: time $O(n)$, space $O(1)$

```
Left = 0
Right = len(s)-1
While lefts < right:
    s[left], s[right] = s[right], s[left]
    Left += 1
    Right -= 1
Return '.join(s)
```
 - Recursion: Time $O(n)$, space $O(n)$

```
Def reverseStringRec(arr, l, r):
    If l >= r:
        Return
    Arr[l], arr[r] = arr[r], arr[l]
    reverseStringRec(arr, l+1, r-1)

reverseStringRec(arr, 0, len(arr)-1)
```
 - Using stack: Time $O(n)$, space $O(n)$

```
Stack = []
For char in s:
    Stack.append(char)
Rev = [""] * len(s)
For i in range(len(s)):
    Rev[i] = stack.pop()
```
 - Built-in methods: $S[::-1]$
 - o Rotate a string:
 - Naive approach: time $O(n*d)$, space $O(n)$

```

S = list(s)
N = len(s)
For _ in range(d):
    First = s[0]
    For i in range(n-1):
        s[i] = s[i+1]
    s[n-1] = first

```

- Temporary char array: time $O(n)$, space $O(n)$

```

N = len(s)
D = d % n
Temp = [""] * n
For i in range(n-d):
    Temp[i] = s[d+i]
For i in range(d):
    Temp[n-d+1] = s[i]

```

- Juggling algorithm: time $O(n)$. Space $O(n)$

```

Def gcd(a, b):
    While b:
        a, b = b, a % b
    Return a
Def rotate(s, d):
    N = len(s)
    D %= n
    Cycles = gcd(n, d)
    Arr = list(s)
    For i in range(cycles):
        Temp = arr[i]
        J = i
        While True:
            K = (j+d) % n
            If k == i:
                Break
            Arr[j] = arr[k]
            J = k
        Arr[j] = temp

```

- Reversal algorithm: time $O(n)$, space $O(n)$

```

N = len(s)
D %= n
Temp = list(s)
Reverse(temp, 0, d-1)
Reverse(temp, d, n-1)
Reverse(temp, 0, n-1)
Def reverse(temp, start, end):
    While start < end:
        Temp[start], temp[end] = temp[end], temp[start]
        Start += 1
        End -= 1

```

- Checks for palindrome

- Two-pointers: time $O(n)$, space $O(1)$

```

Left = 0
Right = len(s)-1
While left < right:
    If s[left] != s[right]:
        Return 0
    Left += 1
    Right -= 1
Return 1

```

- Using single variable: time $O(n)$, space $O(1)$

```

Length = len(s)
For I in range(length // 2):
    If s[i] != s[length-i-1]:
        Return 0
Return 1

```

- Recursion: time $O(n)$, space $O(n)$

```

Def is_palindrom_util(s, left, right):
    If left >= right:
        Return 1
    If s[left] != s[right]:
        Return 0
    Return is_palindrome_util(s, left+1, right-1)

```

- By reversing string: time $O(n)$, space $O(n)$

```

Def is_palindrome(s):
    Return 1 if s == s[::-1] else 0

```

MATRIX

```

rows = 3
cols = 3
rows, cols = (3, 3)
arr = [[0]*cols]*rows
print(arr)

```

```
arr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
```

Operations:

- Access elements of matrix:


```

arr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print("First element of first row:", arr[0][0])
print("Third element of second row:", arr[1][2])
print("Second element of third row:", arr[2][1])

```
- Traversal of a matrix


```

arr = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
]
for row in arr:
    for x in row:
        print(x, end=" ")
    print()

```
- Searching


```

def search_in_matrix(arr, x):
    rows, cols = len(arr), len(arr[0])
    for i in range(rows):
        for j in range(cols):
            if arr[i][j] == x:
                return True
    return False

```

```

x = 8
arr = [
    [0, 6, 8, 9, 11],
    [20, 22, 28, 29, 31],
    [36, 38, 50, 61, 63],
    [64, 66, 100, 122, 128]
]

```

```
]
```

```
if search_in_matrix(arr, x):  
    print("YES")  
else:  
    print("NO")
```

- Sorting

- Row-wise: time $O(r*c*\log c)$, space $O(1)$

```
def sortRows(mat):  
    for row in mat:  
        row.sort()
```

```
mat = [  
    [77, 11, 22, 3],  
    [11, 89, 1, 12],  
    [32, 11, 56, 7],  
    [11, 22, 44, 33]  
]
```

```
sortRows(mat)
```

```
print('\n', end="")  
for row in mat:  
    print(' ', end="")  
    print(' '.join(map(str, row)), end="")  
    print('\n')
```

- Column-wise: first traverse the matrix, then find the transpose, store it in a 2D vector, traverse the rows of the 2D vector, sort each row, store again the 2D vector in the original matrix. Time $O(n*m)$, space $O(n*m)$

Python3 program for the above approach

```
# Function to find the transpose  
# of the matrix mat[]  
def transpose(mat, row, col):
```

```
    # Stores the transpose  
    # of matrix mat[][]  
    tr = [[0 for i in range(row)] for i in range(col)]
```

```
    # Traverse each row of the matrix  
    for i in range(row):
```

```
        # Traverse each column of the matrix  
        for j in range(col):
```

```
            # Transpose matrix elements  
            tr[j][i] = mat[i][j]  
    return tr
```

```
# Function to sort the given  
# matrix in row wise manner  
def RowWiseSort(B):
```

```
    # Traverse the row  
    for i in range(len(B)):
```

```
        # Row - Wise Sorting
```

```

        B[i] = sorted(B[i])
    return B

# Function to print the matrix
# in column wise sorted manner
def sortCol(mat, N, M):
    # Function call to find transpose
    # of the matrix mat[][]
    B = transpose(mat, N, M)

    # Sorting the matrix row-wise
    B = RowWiseSort(B)

    # Calculate transpose of B[][]
    mat = transpose(B, M, N)

    # Print the matrix mat[][]
    for i in range(N):
        for j in range(M):
            print(mat[i][j], end = " ")
        print()

# Driver Code
if __name__ == '__main__':
    # Input
    mat = [ [1, 6, 10 ],
            [ 8, 5, 9 ],
            [ 9, 4, 15 ],
            [ 7, 3, 60 ] ]

    N = len(mat)
    M = len(mat[0])

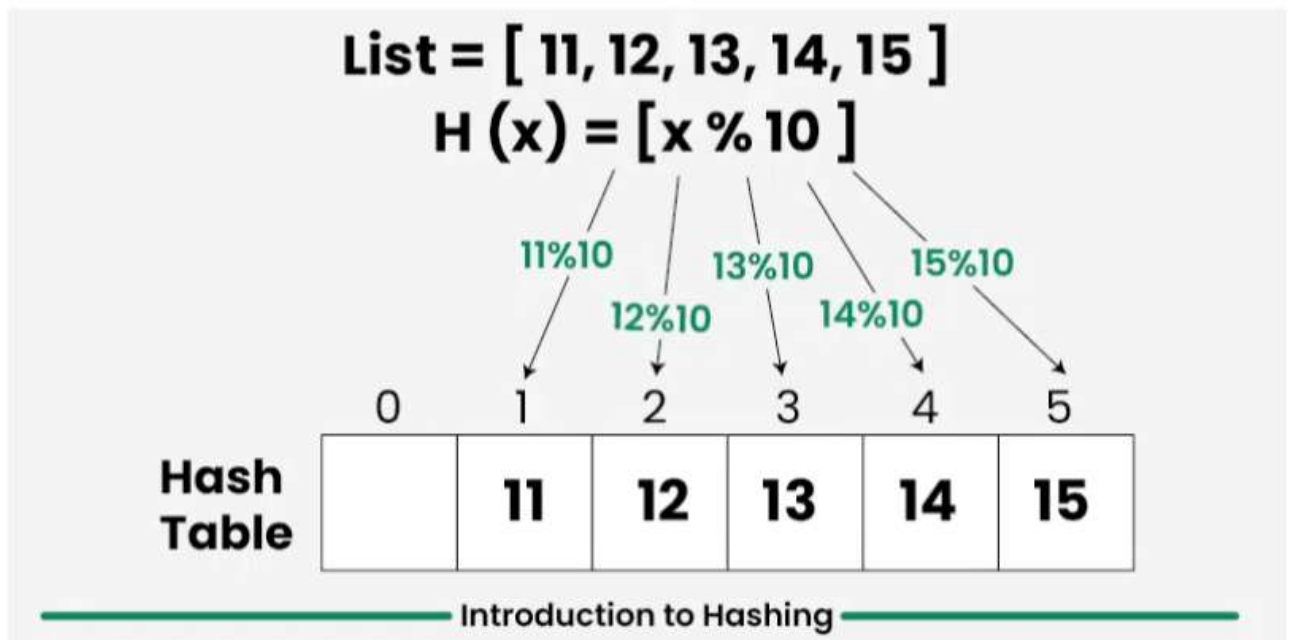
    # Function call to print the matrix
    # in column wise sorted manner
    sortCol(mat, N, M)

# This code is contributed by mohit kumar 29.

```

HASHING

It refers to the process of generating small sized output from an input of typically large and variable size. Hashing uses mathematical formulas known as hash functions to do the transformation. This technique determines an index or location for the storage of an item in a data structure called hash table.



There are two forms of hash:

- Hash set: collection of unique keys
- Hash map: collection of key value pairs with keys being unique

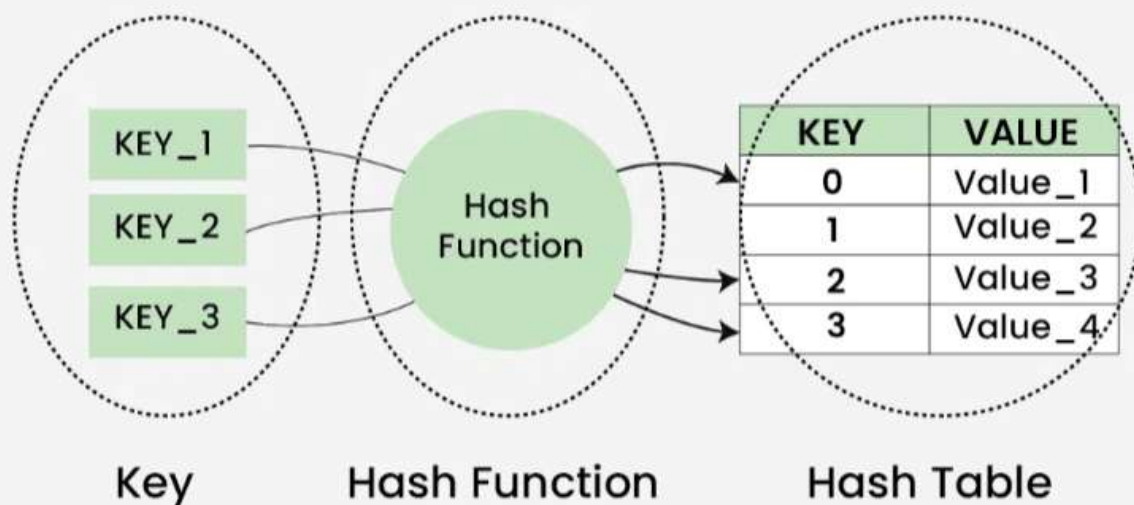
Hash is not used:

- Need to maintain sorted data along with search, insert and delete. We use a self-balancing BST in these cases.
- When strings are keys and we need operations like prefix search along with search, insert and delete.
- When we need operations like floor and ceiling along with search, insert and/or delete.

Components:

- Key: can be an integer or a string which is fed as input in the hash function
- Hash function: receives the input key and returns the index of an element in the array called hash table.
- Hash table: array of lists. It stores values corresponding to the keys.

Components of Hashing



Python has a built-in hash function. You usually use a dict or a set. But if it is specifically asked to build a Hash Table with chaining from scratch:

class HashNode:

```
def __init__(self, key, value):
    self.key = key
    self.value = value
    self.next = None
```

class HashTable:

```
def __init__(self, capacity=97):
    self.capacity = capacity
    self.table = [None] * self.capacity
```

```
def _hash(self, key):
    return hash(key) % self.capacity
```

```
def insert(self, key, value):
    index = self._hash(key)
    head = self.table[index]

    # Update if exists
    curr = head
    while curr:
        if curr.key == key:
            curr.value = value
            return
        curr = curr.next

    # Insert at head
    new_node = HashNode(key, value)
    new_node.next = self.table[index]
    self.table[index] = new_node
```

```
def search(self, key):
    index = self._hash(key)
    curr = self.table[index]
    while curr:
        if curr.key == key:
            return curr.value
        curr = curr.next
    return None
```

In C we have two ways: Linear Chaining and Open Addressing. In the first case, in python, we use a list of lists (or a list of Node).

class Node:

```
def __init__(self, key, value):
    self.key = key
    self.value = value
    self.next = None
```

class ChainingHashTable:

```
def __init__(self, size=97):
    self.size = size
    self.table = [None] * self.size
```

```
def _hash(self, key):
    return hash(key) % self.size
```

```
def insert(self, key, value):
    idx = self._hash(key)
    # Check if key exists to update (like C implementation)
    curr = self.table[idx]
    while curr:
        if curr.key == key:
            curr.value = value
            return
        curr = curr.next
    # Else, insert new at head (Chaining)
    new_node = Node(key, value)
    new_node.next = self.table[idx]
    self.table[idx] = new_node
```

In the second case, because python lists are dynamic, we treat the array as a fixed-size buffer filled with None or a special deleted marker if needed.

Hash Set

class OpenAddressingHashTable:

```
def __init__(self, size=97):
    self.size = size
    self.table = [None] * self.size
    self.DELETED = object() # Marker for lazy deletion
```

```
def _hash(self, key):
    return hash(key) % self.size
```

```
def insert(self, key, value):
    idx = self._hash(key)
```

Linear Probing

```
for i in range(self.size):
```

```
    probe_idx = (idx + i) % self.size
```

```
    # If empty or marked as deleted, insert here
```

```
    if self.table[probe_idx] is None or self.table[probe_idx] is self.DELETED:
```

```

        self.table[probe_idx] = (key, value)
        return
    # If key matches, update
    if self.table[probe_idx][0] == key:
        self.table[probe_idx] = (key, value)
        return
    raise Exception("Hash Table Overflow")

def search(self, key):
    idx = self._hash(key)
    for i in range(self.size):
        probe_idx = (idx + i) % self.size
        if self.table[probe_idx] is None:
            return None # Not found
        if self.table[probe_idx] is not self.DELETED and self.table[probe_idx][0] == key:
            return self.table[probe_idx][1]
    return None

s = {10, 50, 20}
print(s)
print(type(s))

# typecasting list to set
s = set(["a", "b", "c"])
print(s)

# Adding element to the set
s.add("d")
print(s)

s = {"Geeks", "for", 10, 52.7, True}
print(s)

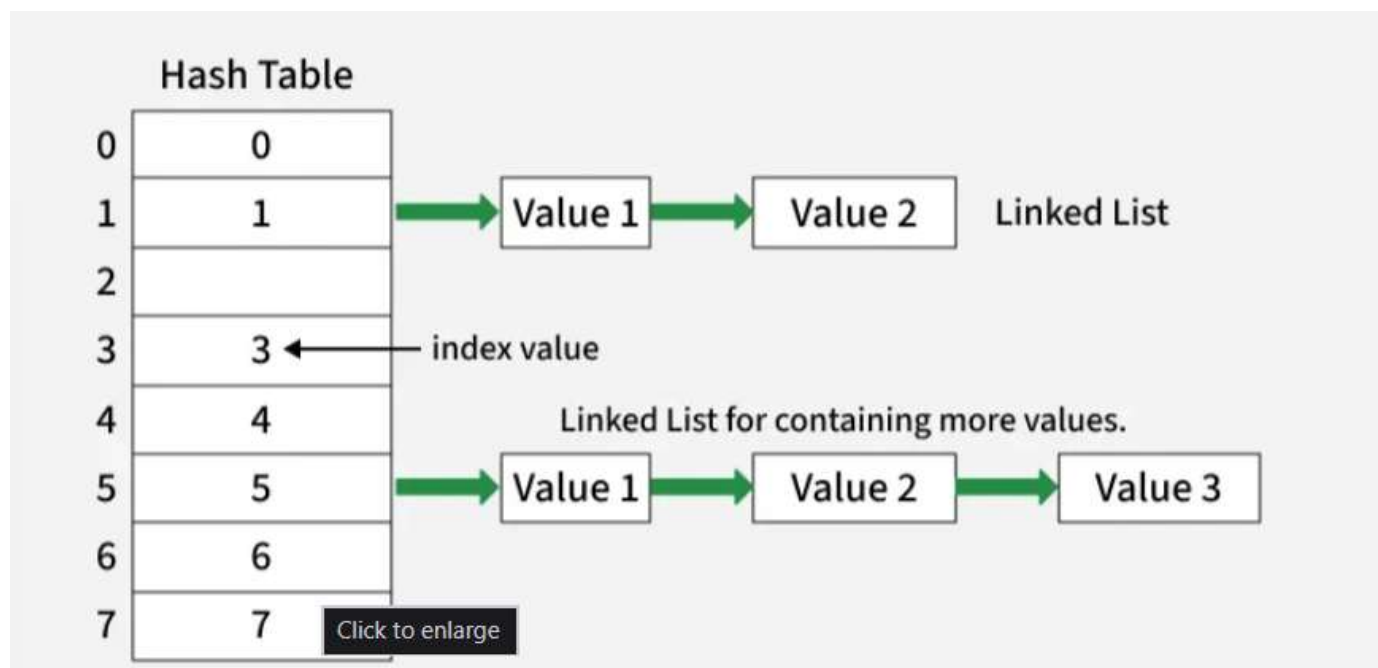
A frozen set is an immutable version of a set.
# Normal set (mutable)
s = set(["a", "b", "c"])
print("Normal Set:", s)

# Frozen set (immutable)
fs = frozenset(["e", "f", "g"])
print("Frozen Set:", fs)

```

If Multiple values are present at the same index position, then the value is appended to that index position, to form a Linked List.

In, Python Sets are implemented using a dictionary with dummy variables, where key beings the members set with greater optimizations to the time complexity.



Methods:

- Adding elements: add
- Union: union
- Intersection: intersection
- Difference: difference
- Clearing: clear

Operators	Notes
key in s	containment check
key not in s	non-containment check
s1 == s2	s1 is equivalent to s2
s1 != s2	s1 is not equivalent to s2
s1 <= s2	s1 is subset of s2
s1 < s2	s1 is proper subset of s2
s1 >= s2	s1 is superset of s2
s1 > s2	s1 is proper superset of s2
s1 s2	the union of s1 and s2
s1 & s2	the intersection of s1 and s2
s1 - s2	the set of elements in s1 but not s2
s1 ^ s2	the set of elements in precisely one of s1 or s2

Hash Map

```
data = { "name": "Jake", "age": 22 }
print(data)
```

```
a = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print(a)
```

```
b = dict(a = "Geeks", b = "for", c = "Geeks")
print(b)
```

```
d = { "name": "Kat", 1: "Python", (1, 2): [1,2,4] }
```

```
# Access using key
print(d["name"])
```

```
# Access using get()
print(d.get("name"))
```

Methods:

- Removing dictionary items:

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks', 'age': 22}
```

```
# Using del
del d["age"]
print(d)
```

```
# Using pop()
val = d.pop(1)
print(val)
```

```
# Using popitem()
key, val = d.popitem()
print(f"Key: {key}, Value: {val}")
```

```
# Using clear()
d.clear()
print(d)
```

- Iterate:

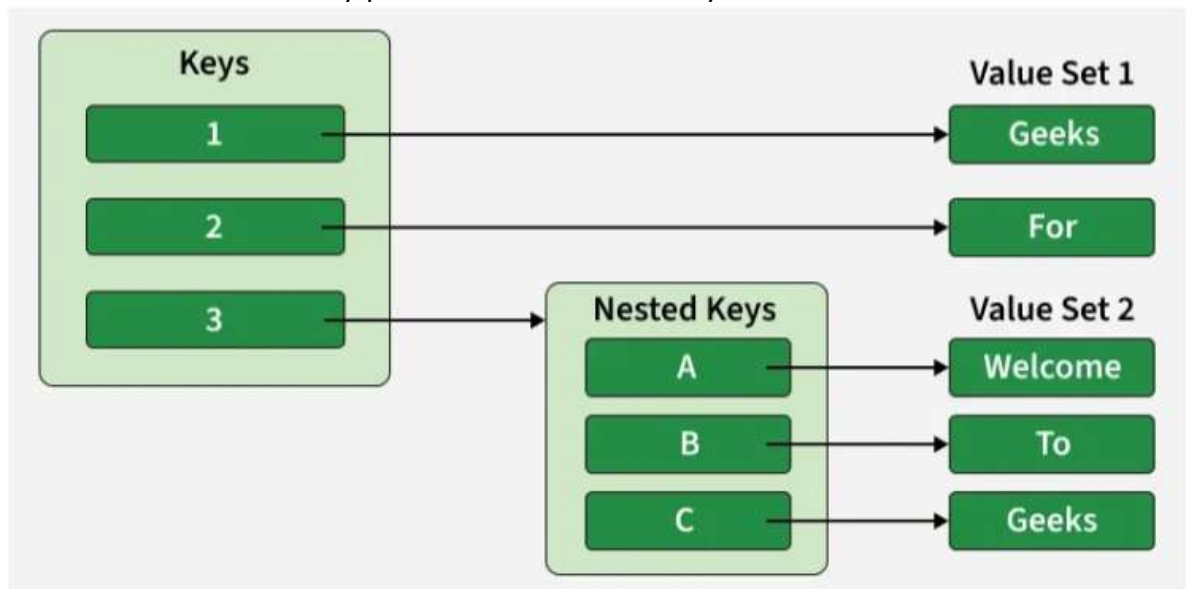
```
d = {1: 'Geeks', 2: 'For', 'age': 22}
```

```
# Iterate over keys
for key in d:
    print(key)
```

```
# Iterate over values
for value in d.values():
    print(value)
```

```
# Iterate over key-value pairs
for key, value in d.items():
    print(f"{key}: {value}")
```

- Nested dictionaries: each key points to another dictionary.



```
d = {1: 'Geeks', 2: 'For', 3: {'A': 'Welcome', 'B': 'To', 'C': 'Geeks'}}
print(d)
```

Bucket Sort

Each number in the array appears a certain number of times, and the maximum possible

frequency is the length of the array.

Create n empty buckets :

- Insert `arr[i]` into `bucket[n*arr[i]]`
- Sort individual buckets using insertion sort
- Concatenate all sorted buckets

Inserting elements into the buckets:

- Multiply each element by the size of the bucket array
- Convert the result to an integer and we obtain the index of the vector.
- Insert the element into the bucket corresponding to the calculated index.
- Repeat for all the inputs.

Iterate through each bucket in order and insert each individual element from the bucket into the original array.

Time complexity : $O(n^2)$ in the worst case and $O(n+k)$ in the best case

- Build a frequency map that counts how many times each number appears.
- Create a list of groups freqs, where each position will store all numbers to `freq[frequency]`
- Loop from the largest frequency to 1 and for each number in `freq[i]`, add it to the result list. Once the result contains k numbers, return it.

defaultDict in Python

It automatically assigns a default value to keys that do not exist which means no need to manually check for missing values and void `KeyError`.

from collections import defaultdict

d = defaultdict(list)

```
d['fruits'].append('apple')
d['vegetables'].append('carrot')
print(d)
print(d['juices'])
```

Syntax: `defaultdict(default_factory)`. `Default_factory` is a callable (list, int, set, str) that provides the default value for missing keys.

Tuples

A tuple is an immutable ordered collection of elements. Tuples are similar to lists but they cannot change after creation. They can hold data of different types and are ordered.

```
tup = ()
print(tup)
```

```
# Using String
tup = ('Geeks', 'For')
print(tup)
```

```
# Using List
li = [1, 2, 4, 5, 6]
print(tuple(li))
```

```
# Using Built-in Function
tup = tuple('Geeks')
print(tup)
tup = (5, 'Welcome', 7.5, True, [1, 2, 3], {'key': 'value'})
print(tup)
```

Basic operations:

- Accessing tuples

```

tup = tuple("Geeks")
print(tup[0])
print(tup[1:4])
print(tup[:3])

# Tuple unpacking
tup = ("Geeks", "For", "Geeks")

# This line unpack values of Tuple1
a, b, c = tup
print(a)
print(b)
print(c)
    - Concatenation of tuples
tup1 = (0, 1, 2, 3)
tup2 = ('Geeks', 'For', 'Geeks')
tup3 = tup1 + tup2
print(tup3)
    - Slicing of tuples: returns a shallow copy
tup = tuple('GEEKSFORGEEKS')
print(tup[1:])
print(tup[::-1])
print(tup[4:9])
    - Deleting a tuple
tup = (0, 1, 2, 3, 4)
del tup
print(tup)
    - Tuple unpack with *: the * means collecting everything into a list
tup = (1, 2, 3, 4, 5)
a, *b, c = tup
print(a)
print(b)
print(c)

```

Ordered Dict

It remembers the order in which keys are inserted. It provides:

- Order-sensitive equality check
- Easy implementation of data structures like queues, stacks and LRU caches.

```

from collections import OrderedDict

```

```

od = OrderedDict()
od['apple'] = 1
od['banana'] = 2
od['cherry'] = 3

```

```

print(list(od.items()))

```

The difference between OrderedDict and Dict is that only the first one gives advance control over order, even if both preserve insertion order.

```

from collections import OrderedDict

```

```

print("dict")
d = {}
d['a'] = 1

```

```
d['b'] = 2
d['c'] = 3
d['d'] = 4
for key, val in d.items():
    print(key, val)
```

```
print("ordered dict")
od = OrderedDict()
od['d'] = 4
od['b'] = 2
od['a'] = 1
od['c'] = 3
for key, val in od.items():
    print(key, val)
```

Key features:

1. Insertion order preservation
2. Changing values does not affect order.

```
from collections import OrderedDict
```

```
od = OrderedDict([('a', 1), ('b', 2), ('c', 3), ('d', 4)])
od['c'] = 5
```

```
for k, v in od.items():
    print(k, v)
```

3. Equality check consider order

```
from collections import OrderedDict
```

```
od1 = OrderedDict([('a', 1), ('b', 2), ('c', 3)])
od2 = OrderedDict([('c', 3), ('b', 2), ('a', 1)])
print(od1 == od2)
```

4. Reversing an OrderedDict

```
from collections import OrderedDict
```

```
d1 = OrderedDict([('a', 1), ('b', 2), ('c', 3)])
d2 = OrderedDict(reversed(list(d1.items())))
```

```
for k, v in d2.items():
    print(k, v)
```

5. Pop last or first item

```
from collections import OrderedDict
```

```
d = OrderedDict([('a', 1), ('b', 2), ('c', 3)])
```

```
res = d.popitem(last=True) # Remove last inserted item
print(res)
```

6. Move keys to front or end

```
from collections import OrderedDict
```

```
d = OrderedDict([('a', 1), ('b', 2), ('c', 3)])
```

```
d.move_to_end('a') # Move 'a' to end
d.move_to_end('b', last=False) # Move 'b' to front
```

```
for k, v in d.items():
    print(k, v)
```

7. Deleting and re-inserting keys

```
from collections import OrderedDict
od = OrderedDict([('a', 1), ('b', 2), ('c', 3), ('d', 4)])
od.pop('c') # Delete 'c'
```

```
for k, v in od.items():
    print(k, v)
```

```
od['c'] = 3 # Re-insert 'c' at end
for k, v in od.items():
    print(k, v)
```

Feature	Dict	OrderedDict
Maintains insertion order	Yes	Yes
Allows key reordering	No	Yes (move_to_end(key, last=True))
Pop items from ends	No (only popitem() removes last item)	Yes (popitem(last=True/False) supports both ends)
Equality check considers order	No (order ignored)	Yes (order matters)
Performance	Faster	Slightly slower